

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see [jupyter.org](#).

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
# =====
# Quantization and Image Detail Demonstration
# Run this cell in Google Colab to visualize the mathematical impact of bit-depth
# allocation on image fidelity, false contouring, and quantization noise.
# =====

import numpy as np
import matplotlib.pyplot as plt

# 1. Generate a synthetic smooth grayscale gradient (representing a high-resolution continuous signal)
width, height = 600, 400
# Create a 2D linear gradient ramp from 0 (black) to 255 (white)
gradient_line = np.linspace(0, 255, width, dtype=np.float32)
analog_signal = np.tile(gradient_line, (height, 1))

# 2. Define the mathematical Quantization Function
def quantize_image(img, bit_depth):
    """
    Quantizes an image to a target bit-depth.
    Maps continuous scale [0, 255] down to 2^bit_depth discrete intervals,
    rounds the values, and scales back to [0, 255] for visual comparison.
    """
    levels = 2 ** bit_depth
    # Step 1: Scale to [0, levels - 1]
    scaled = img * (levels - 1) / 255.0
```

```

# Step 2: Round to nearest integer (Quantization step)
quantized_discrete = np.round(scaled)
# Step 3: Re-scale back to [0, 255] range for uniform display
reconstructed = quantized_discrete * (255.0 / (levels - 1))
return reconstructed

# 3. Process the signal through various Bit-Depth allocations
bit_depths = [8, 4, 3, 2]
results = {}

for b in bit_depths:
    quantized_img = quantize_image(analog_signal, b)
    # Calculate Mean Squared Error (Quantization Noise Floor)
    mse = np.mean((analog_signal - quantized_img) ** 2)
    results[b] = {
        'image': quantized_img,
        'mse': mse
    }

# 4. Visualization Setup using Matplotlib
fig, axes = plt.subplots(2, 2, figsize=(14, 10), dpi=100)
axes = axes.ravel()

for i, b in enumerate(bit_depths):
    img_data = results[b]['image']
    error_mse = results[b]['mse']
    levels = 2 ** b

    # Plot the quantized image
    im = axes[i].imshow(img_data, cmap='gray', vmin=0, vmax=255)

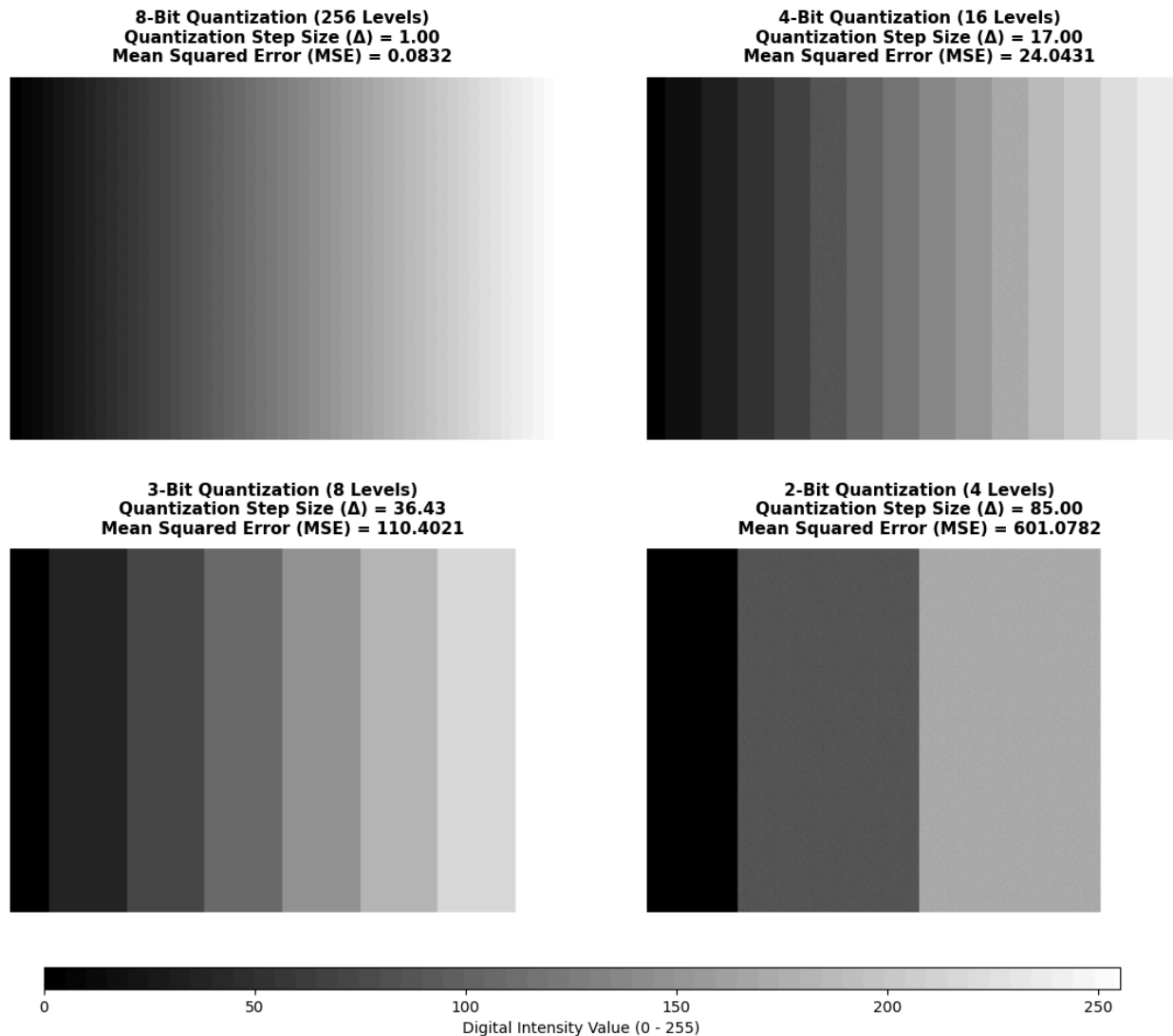
    # Format labels to reflect technical metrics
    axes[i].set_title(
        f"{b}-Bit Quantization ({levels} Levels)\n"
        f"Quantization Step Size ( $\Delta$ ) = {255/(levels-1):.2f}\n"
        f"Mean Squared Error (MSE) = {error_mse:.4f}",
        fontsize=11,
        fontweight='bold',
        pad=10
    )
    axes[i].axis('off')

# Add a colorbar at the bottom to illustrate structural transitions
cbar_ax = fig.add_axes([0.15, 0.05, 0.7, 0.02])
fig.colorbar(im, cax=cbar_ax, orientation='horizontal', label='Digital Intensity Value (0 - 255)')

plt.suptitle(
    "Visualizing the Impact of Quantization & False Contouring",
    fontsize=16,
    fontweight='bold',
    y=0.98
)
plt.subplots_adjust(hspace=0.3, wspace=0.15, bottom=0.12)
plt.show()

```

Visualizing the Impact of Quantization & False Contouring



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs](#) and [TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the [10 minute guide](#) or [US stock market data analysis demo](#).

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just [a few lines of code](#).

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the [machine learning examples](#) below.